

Prompt - Building a File Anonymizer with Claude Code

I want you to build a local Python app for me that has a nice dashboard running in my chrome browser, where I can upload ANY common text file or files (pdf, txt, powerpoint, word and so on) and the app uses Azure Open AI GPT4o mini to strip any personal data from the documents. When the app is done it should automatically download all files in the form of pdfs (no matter the original file type) to my computer. Here are the details:

- Create the Frontend in the same design and style as the revenue dashboard
- The App has to work with one file, mixed files types and also multiple files of the same type
- After the personal data is stripped and replaced with suitable place holders the files have to be downloaded as individual pdfs independent of the original file type
- This is the personal data that should be stripped: Names, Dates, Phone Numbers, Email Addresses, Addresses, Company Names, School and University Names
- The name of the download files should be the same as the names of the uploaded files but with "cleaned" attached.
- The app should have a nice loading bar or spinner, while the LLM is working.
- The app should be easily startable with one click, ideally we have a desktop icon or so.
- An Azure OpenAI API Key, the Azure Endpoint URL, the deployment name and the API version can be found in .env.

Here you see how to work with Azure OpenAI LLMs using Python. The key and URI to the azure openAI gpt4o mini model is also in .env:

Erste Schritte

Nachfolgend finden Sie Beispielcodeausschnitte für einige Anwendungsfälle. Weitere Informationen zum Azure OpenAI SDK finden Sie in der vollständigen [Dokumentation](#) und in [Beispielen](#).

1. Authentifizierung mit API-Schlüssel

Stellen Sie für OpenAI-API-Endpunkte das Modell bereit, um die Endpunkt-URL und einen API-Schlüssel zur Authentifizierung beim Dienst zu generieren. In diesem Beispiel sind der Endpunkt und der Schlüssel Zeichenfolgen, die die Endpunkt-URL und den API-Schlüssel enthalten.

Die API-Endpunkt-URL und den API-Schlüssel finden Sie auf der Seite „Bereitstellungen + Endpunkt“, sobald das Modell bereitgestellt wurde.

Um einen Client mit dem OpenAI SDK unter Verwendung eines API-Schlüssels zu erstellen, initialisieren Sie den Client, indem Sie Ihren API-Schlüssel an die SDK-Konfiguration übergeben. Dadurch können Sie sich authentifizieren und nahtlos mit den Diensten von OpenAI interagieren.

```
import os
from openai import AzureOpenAI
```

```
client = AzureOpenAI(
    api_version="2024-12-01-preview",
    azure_endpoint="https://azuredom.openai.azure.com/",
    api_key=subscription_key,
)
```

2. Abhängigkeiten installieren

Installieren Sie das Azure Open AI-SDK mit pip (Erfordert: Python >=3.8):
`pip install openai`

3. Einfachen Beispielcode ausführen

Dieses Beispiel zeigt einen grundlegenden Aufruf der Chatabschluss-API. Der Aufruf ist synchron.

```
import os
from openai import AzureOpenAI

endpoint = "https://azuredom.openai.azure.com/"
model_name = "gpt-4o-mini"
deployment = "gpt-4o-mini"

subscription_key = "<your-api-key>"
api_version = "2024-12-01-preview"

client = AzureOpenAI(
    api_version=api_version,
    azure_endpoint=endpoint,
    api_key=subscription_key,
)

response = client.chat.completions.create(
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant.",
        },
        {
            "role": "user",
            "content": "I am going to Paris, what should I see?",
        }
    ],
    max_tokens=4096,
    temperature=1.0,
    top_p=1.0,
    model=deployment
)

print(response.choices[0].message.content)
```

4. Weitere Beispiele erkunden

Ausführen einer mehrteiligen Unterhaltung

Dieses Beispiel zeigt eine Unterhaltung mit mehreren Gesprächen mit der Chatvervollständigungs-API. Wenn Sie das Modell für eine Chatanwendung verwenden, müssen Sie den Verlauf dieser Unterhaltung verwalten und die neuesten Nachrichten an das Modell senden.

```
import os
from openai import AzureOpenAI

endpoint = "https://azuredom.openai.azure.com/"
model_name = "gpt-4o-mini"
deployment = "gpt-4o-mini"

subscription_key = "<your-api-key>"
api_version = "2024-12-01-preview"

client = AzureOpenAI(
```

```

    api_version=api_version,
    azure_endpoint=endpoint,
    api_key=subscription_key,
)

response = client.chat.completions.create(
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant.",
        },
        {
            "role": "user",
            "content": "I am going to Paris, what should I see?",
        },
        {
            "role": "assistant",
            "content": "Paris, the capital of France, is known for its stunning architecture, art museums, historical landmarks, and romantic atmosphere. Here are some of the top attractions to see in Paris:\n\n1. The Eiffel Tower: The iconic Eiffel Tower is one of the most recognizable landmarks in the world and offers breathtaking views of the city.\n2. The Louvre Museum: The Louvre is one of the worlds largest and most famous museums, housing an impressive collection of art and artifacts, including the Mona Lisa.\n3. Notre-Dame Cathedral: This beautiful cathedral is one of the most famous landmarks in Paris and is known for its Gothic architecture and stunning stained glass windows.\n\nThese are just a few of the many attractions that Paris has to offer. With so much to see and do, its no wonder that Paris is one of the most popular tourist destinations in the world.",
        },
        {
            "role": "user",
            "content": "What is so great about #1?",
        }
    ],
    max_tokens=4096,
    temperature=1.0,
    top_p=1.0,
    model=deployment
)

print(response.choices[0].message.content)

```

Streamen der Ausgabe

Für eine bessere Benutzererfahrung sollten Sie die Antwort des Modells streamen, damit das erste Token frühzeitig angezeigt wird und Sie nicht auf lange Antworten warten müssen.

import os

from openai import AzureOpenAI

endpoint = "https://azuredom.openai.azure.com/"

model_name = "gpt-4o-mini"

deployment = "gpt-4o-mini"

subscription_key = "<your-api-key>"

api_version = "2024-12-01-preview"

```
client = AzureOpenAI(
    api_version=api_version,
    azure_endpoint=endpoint,
    api_key=subscription_key,
)

response = client.chat.completions.create(
    stream=True,
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant.",
        },
        {
            "role": "user",
            "content": "I am going to Paris, what should I see?",
        }
    ],
    max_tokens=4096,
    temperature=1.0,
    top_p=1.0,
    model=deployment,
)

for update in response:
    if update.choices:
        print(update.choices[0].delta.content or "", end="")

client.close()
```